ОТЧЕТ ПО ДИСЦИПЛИНЕ

"ИНФОРМАТИКА "

"ОСНОВЫ АЛГОРИТМИЗАЦИИ И ПРОГРАММИРОВАНИЯ "

"ВВЕДЕНИЕ В ИНЖЕНЕРНУЮ ДЕЯТЕЛЬНОСТЬ "

Лабораторная работа № 7
7.1. "Перегрузка функций в Си++"
ВАРИАНТ 14

ВЫПОЛНИЛА:

Студентка группы РИС-26-3Б

Мальцева Альбина Витальевна

ПРОВЕРИЛА:

Доцент кафедры ИТАС

О.А.Полякова

2026

ПОСТАНОВКА ЗАДАЧИ

Написать перегруженные функции и основную программу, которая их вызывает. Согласно варианту 14:

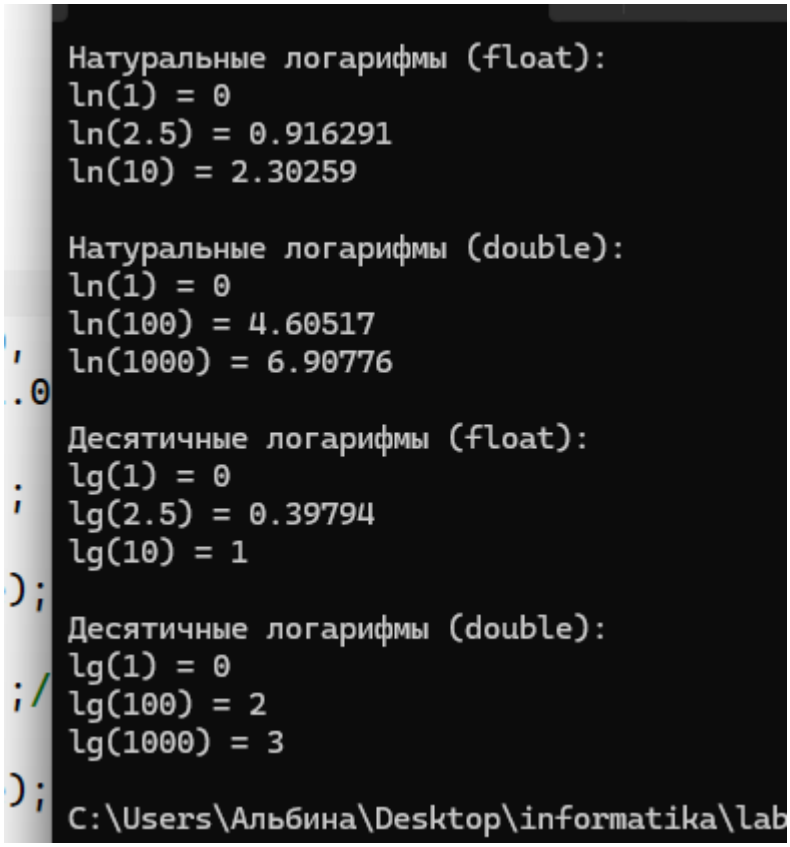
- **а)** функция для вычисления натурального логарифма;
- **б)** функция для вычисления десятичного логарифма.

ТЕКСТ ПРОГРАММЫ

```
#include <iostream>
#include <cmath>
using namespace std;
void natural_log(int n, float a[])   // Принимает размер n и массив float
{
   cout << "Натуральные логарифмы (float):" << endl;
   for (int i = 0; i < n; i++)// Проходим по всем n элементам массива
   {
      cout << "ln(" << a[i] << ") = " << log(a[i]) << endl;
   }
}
//вычисляет натуральный логарифм для массива типа double
void natural_log(int n, double a[]) // Принимает размер n и массив double
{
   cout << "Натуральные логарифмы (double):" << endl;
   for (int i = 0; i < n; i++)// Проходим по всем n элементам массива
   {
      cout << "ln(" << a[i] << ") = " << log(a[i]) << endl; // Печатаем ln(a[i]) через функцию log()
   }
}
//вычисляет десятичный логарифм для массива типа float
void decimal_log(int n, float a[])// Принимает размер n и массив float
{
   cout << "Десятичные логарифмы (float):" << endl; // Печатаем заголовок
   for (int i = 0; i < n; i++)// Проходим по всем n элементам массива
   {
      cout << "lg(" << a[i] << ") = " << log10(a[i]) << endl; // Печатаем lg(a[i]) через функцию log10()
   }
}
//вычисляет десятичный логарифм для массива типа double
void decimal_log(int n, double a[])// Принимает размер n и массив double
{
   cout << "Десятичные логарифмы (double):" << endl;
   for (int i = 0; i < n; i++)// Проходим по всем n элементам массива
   {
      cout << "lg(" << a[i] << ") = " << log10(a[i]) << endl; // Печатаем lg(a[i]) через функцию log10()
   }
}
int main()
{
   float mas_float[] = { 1.0, 2.5, 10.0 };      // Объявляем массив типа float с тремя значениями
   double mas_double[] = { 1.0, 100.0, 1000.0 }; // Объявляем массив типа double с тремя значениями

   natural_log(3, mas_float); // Вызываем natural_log для float (компилятор выберет версию с float)
   cout << endl;
   natural_log(3, mas_double);// Вызываем natural_log для double (компилятор выберет версию с double)
```

```
    cout << endl;
    decimal_log(3, mas_float);// Вызываем decimal_log для float (компилятор выберет версию с float)
    cout << endl;
    decimal_log(3, mas_double); // Вызываем decimal_log для double (компилятор выберет версию с double)
    return 0;
}
```



ОБЪЯСНЕНИЕ

1. **Перегрузка функций** — это механизм языка C++, который позволяет создавать несколько функций с **одинаковым именем**, но разными параметрами.
2. В программе объявлены две функции с именем natural_log и две функции с именем decimal_log. Они отличаются типом принимаемого массива (float или double).
3. В функции main при вызове natural_log(3, mas_float) компилятор анализирует тип второго аргумента (mas_float имеет тип float[]) и автоматически выбирает правильную версию функции — ту, которая принимает float. При вызове natural_log(3, mas_double) — выбирается версия для double.
4. Вычисления производятся с помощью стандартных математических функций из библиотеки <cmath>: log() для натурального логарифма и log10() для десятичного.
5. Результаты полностью соответствуют математическим правилам:
    a. натуральный логарифм от 1 равен 0 (так как *e0=1e*0=1);
    b. десятичный логарифм от 100 равен 2 (так как *102=100*102=100);
    c. десятичный логарифм от 1000 равен 3 (так как *103=1000*103=1000).

ВЫВОД

В ходе выполнения лабораторной работы №7.1 были изучены и применены на практике механизмы **перегрузки функций** в языке C++.

В соответствии с вариантом 14 были разработаны перегруженные функции для вычисления натурального (natural_log) и десятичного (decimal_log) логарифмов. Перегрузка была реализована путем создания двух версий каждой функции: одна версия работает с массивами типа float, а другая — с массивами типа double.

В ходе работы было установлено, что перегрузка функций позволяет использовать одно и то же имя для логически схожих операций, что делает код более читаемым и удобным. Компилятор автоматически определяет, какую именно версию функции необходимо вызвать, основываясь на типах фактических параметров, переданных при вызове.

Таким образом, задача выполнена в полном объеме: программа корректно обрабатывает исходные данные, наглядно демонстрирует работу механизма перегрузки функций и выводит результаты вычислений, полностью соответствующие математическим законам.